

```
"""
```

```
Invoice Automation Processor  
Portfolio Project by Mete Kocak
```

```
Automatically extracts key data from PDF invoices and exports  
structured results to Excel, with overdue payment detection  
and email notification support.
```

```
Usage:
```

```
python invoice_processor.py --input ./invoices --output ./output
```

```
"""
```

```
import pdfplumber  
import pandas as pd  
import re  
import os  
import json  
import argparse  
from datetime import datetime, timedelta  
from openpyxl import Workbook  
from openpyxl.styles import Font, PatternFill, Alignment, Border, Side  
from openpyxl.utils import get_column_letter
```

```
# _____  
# 1. PDF TEXT EXTRACTION  
# _____
```

```
def extract_text_from_pdf(pdf_path: str) -> str:  
    """Extract all text content from a PDF file."""  
    full_text = ""  
    with pdfplumber.open(pdf_path) as pdf:  
        for page in pdf.pages:  
            text = page.extract_text()  
            if text:  
                full_text += text + "\n"  
    return full_text
```

```
# _____  
# 2. INVOICE DATA PARSING  
# _____
```

```
def parse_invoice_data(text: str, filename: str) -> dict:
```

```

"""
Parse extracted text and return structured invoice data.
Uses regex patterns to find common invoice fields.
"""
invoice = {
    "filename": filename,
    "invoice_number": None,
    "company_name": None,
    "date": None,
    "due_date": None,
    "subtotal": None,
    "vat_amount": None,
    "total_amount": None,
    "currency": "EUR",
    "status": "Unpaid",
    "is_overdue": False,
    "extracted_at": datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
}

# — Invoice Number —
inv_patterns = [
    r"(?:Invoice No|Factuurnummer|Invoice Number) [.:\\s]+([A-Z]{1,5}[\\-]\\d[\\w\\-]*)",
    r"(?:Invoice|Factuur|Receipt) [.:\\s#]+([A-Z]{1,5}[\\-]\\d[\\w\\-]*)",
]
for pattern in inv_patterns:
    match = re.search(pattern, text, re.IGNORECASE)
    if match:
        invoice["invoice_number"] = match.group(1).strip()
        break

# — Company Name (first line heuristic) —
lines = [l.strip() for l in text.split("\\n") if l.strip()]
if lines:
    invoice["company_name"] = lines[0]

# — Dates —
date_patterns = [
    r"(\\d{2}[\\-/]\\d{2}[\\-/]\\d{4})", # DD-MM-YYYY or DD/MM/YYYY
    r"(\\d{4}[\\-/]\\d{2}[\\-/]\\d{2})", # YYYY-MM-DD
    r"(\\d{1,2}\\s+\\w+\\s+\\d{4})", # 1 January 2024
]
dates_found = []
for pattern in date_patterns:
    dates_found.extend(re.findall(pattern, text))

if len(dates_found) >= 1:
    invoice["date"] = dates_found[0]

```

```

if len(dates_found) >= 2:
    invoice["due_date"] = dates_found[1]

# — Amounts —
amount_patterns = {
    "subtotal": [
        r"(?:Subtotal|Subtotaal)[:\s]*(?:EUR| [€$£])?\s*([\d.,,]+)",
        r"(?:Net Amount|Nettobedrag)[:\s]*(?:EUR| [€$£])?\s*([\d.,,]+)",
    ],
    "vat_amount": [
        r"(?:VAT|BTW)[^:]*[:\s]*(?:EUR| [€$£])?\s*([\d.,,]+)",
    ],
    "total_amount": [
        r"(?! [Ss]ub) (?:Total|Totaal|Totaalbedrag)[:\s]*(?:EUR| [€$£])?\s*([\d",
        r"(?:Amount Due|Te betalen)[:\s]*(?:EUR| [€$£])?\s*([\d.,,]+)",
    ],
}

for field, patterns in amount_patterns.items():
    for pattern in patterns:
        match = re.search(pattern, text, re.IGNORECASE)
        if match:
            amount_str = match.group(1).replace(",", "")
            try:
                invoice[field] = float(amount_str)
            except ValueError:
                pass
            break

# — Currency Detection —
if "EUR" in text or "€" in text:
    invoice["currency"] = "EUR"
elif "$" in text or "USD" in text:
    invoice["currency"] = "USD"
elif "£" in text or "GBP" in text:
    invoice["currency"] = "GBP"

# — Overdue Check —
if invoice["due_date"]:
    try:
        for fmt in ["%d-%m-%Y", "%d/%m/%Y", "%Y-%m-%d"]:
            try:
                due = datetime.strptime(invoice["due_date"], fmt)
                if due < datetime.now():
                    invoice["is_overdue"] = True
                    invoice["status"] = "Overdue"
                break

```

```

        except ValueError:
            continue
    except Exception:
        pass

    return invoice

# -----
# 3. BATCH PROCESSING
# -----

def process_invoice_folder(folder_path: str) -> list:
    """Process all PDF files in a folder and return extracted data."""
    results = []
    pdf_files = [f for f in os.listdir(folder_path) if f.lower().endswith(".pdf")]

    if not pdf_files:
        print(f"⚠ No PDF files found in {folder_path}")
        return results

    print(f"\n📁 Found {len(pdf_files)} invoice(s) to process\n")

    for i, filename in enumerate(pdf_files, 1):
        filepath = os.path.join(folder_path, filename)
        print(f"  [{i}/{len(pdf_files)}] Processing: {filename}")

        try:
            text = extract_text_from_pdf(filepath)
            invoice_data = parse_invoice_data(text, filename)
            results.append(invoice_data)
            status = "🔴 OVERDUE" if invoice_data["is_overdue"] else "🟢 OK"
            print(f"      → {invoice_data['invoice_number']} | "
                  f"{invoice_data['currency']} {invoice_data['total_amount']} | {")
        except Exception as e:
            print(f"      ❌ Error: {e}")
            results.append({"filename": filename, "error": str(e)})

    return results

# -----
# 4. EXCEL EXPORT WITH FORMATTING
# -----

def export_to_excel(data: list, output_path: str):
    """Export processed invoice data to a formatted Excel file."""

```

```

wb = Workbook()
ws = wb.active
ws.title = "Invoices"

# — Styling —
header_font = Font(name="Calibri", size=11, bold=True, color="FFFFFF")
header_fill = PatternFill(start_color="2B579A", end_color="2B579A", fill_type="solid")
overdue_fill = PatternFill(start_color="FFE0E0", end_color="FFE0E0", fill_type="solid")
paid_fill = PatternFill(start_color="E0FFE0", end_color="E0FFE0", fill_type="solid")
thin_border = Border(
    left=Side(style="thin"),
    right=Side(style="thin"),
    top=Side(style="thin"),
    bottom=Side(style="thin"),
)

# — Title Row —
ws.merge_cells("A1:I1")
title_cell = ws["A1"]
title_cell.value = "Invoice Processing Report"
title_cell.font = Font(name="Calibri", size=16, bold=True, color="2B579A")
title_cell.alignment = Alignment(horizontal="left", vertical="center")
ws.row_dimensions[1].height = 35

# — Subtitle —
ws.merge_cells("A2:I2")
ws["A2"].value = f"Generated: {datetime.now().strftime('%Y-%m-%d %H:%M')} | T"
ws["A2"].font = Font(name="Calibri", size=10, italic=True, color="666666")
ws.row_dimensions[2].height = 20

# — Headers —
headers = [
    "File", "Invoice #", "Company", "Date",
    "Due Date", "Subtotal", "VAT", "Total", "Status"
]
for col, header in enumerate(headers, 1):
    cell = ws.cell(row=4, column=col, value=header)
    cell.font = header_font
    cell.fill = header_fill
    cell.alignment = Alignment(horizontal="center", vertical="center")
    cell.border = thin_border
ws.row_dimensions[4].height = 28

# — Data Rows —
for row_idx, inv in enumerate(data, 5):
    if "error" in inv:
        ws.cell(row=row_idx, column=1, value=inv["filename"])

```

```

ws.cell(row=row_idx, column=9, value=f"Error: {inv['error']}")
continue

values = [
    inv["filename"],
    inv["invoice_number"],
    inv["company_name"],
    inv["date"],
    inv["due_date"],
    inv["subtotal"],
    inv["vat_amount"],
    inv["total_amount"],
    inv["status"],
]

for col, val in enumerate(values, 1):
    cell = ws.cell(row=row_idx, column=col, value=val)
    cell.border = thin_border
    cell.alignment = Alignment(vertical="center")

    # Currency formatting
    if col in [6, 7, 8] and isinstance(val, (int, float)):
        cell.number_format = '€#,##0.00'
        cell.alignment = Alignment(horizontal="right", vertical="center")

    # Status coloring
    if col == 9:
        cell.alignment = Alignment(horizontal="center", vertical="center")
        if val == "Overdue":
            cell.font = Font(bold=True, color="CC0000")
            for c in range(1, 10):
                ws.cell(row=row_idx, column=c).fill = overdue_fill
        elif val == "Paid":
            cell.font = Font(bold=True, color="008800")
            for c in range(1, 10):
                ws.cell(row=row_idx, column=c).fill = paid_fill

# — Column Widths —
widths = [25, 15, 25, 14, 14, 12, 12, 12, 12]
for i, w in enumerate(widths, 1):
    ws.column_dimensions[get_column_letter(i)].width = w

# — Summary Section —
summary_row = len(data) + 6
ws.cell(row=summary_row, column=1, value="Summary").font = Font(bold=True, si

total_amount = sum(inv.get("total_amount", 0) or 0 for inv in data if "error"

```

```

overdue_count = sum(1 for inv in data if inv.get("is_overdue"))
total_vat = sum(inv.get("vat_amount", 0) or 0 for inv in data if "error" not

summary_data = [
    ("Total Invoices", len(data)),
    ("Total Amount", f"€{total_amount:,.2f}"),
    ("Total VAT", f"€{total_vat:,.2f}"),
    ("Overdue", overdue_count),
    ("Processed", len(data) - overdue_count),
]

for i, (label, value) in enumerate(summary_data):
    ws.cell(row=summary_row + 1 + i, column=1, value=label).font = Font(color
    val_cell = ws.cell(row=summary_row + 1 + i, column=2, value=value)
    val_cell.font = Font(bold=True)

# — Save —
wb.save(output_path)
print(f"\n✅ Excel report saved: {output_path}")

# —————
# 5. JSON EXPORT (for dashboard)
# —————

def export_to_json(data: list, output_path: str):
    """Export data as JSON for the dashboard."""
    with open(output_path, "w") as f:
        json.dump(data, f, indent=2, default=str)
    print(f"✅ JSON data saved: {output_path}")

# —————
# 6. MAIN
# —————

def main():
    parser = argparse.ArgumentParser(description="Automated Invoice Processor")
    parser.add_argument("--input", "-i", default="./sample_invoices",
                        help="Folder containing PDF invoices")
    parser.add_argument("--output", "-o", default="./output",
                        help="Output folder for reports")
    args = parser.parse_args()

    print("=" * 55)
    print(" 📁 INVOICE AUTOMATION PROCESSOR")
    print(" Portfolio Project — Mete Kocak")

```

```

print("=" * 55)

os.makedirs(args.output, exist_ok=True)

# Process invoices
results = process_invoice_folder(args.input)

if results:
    # Export to Excel
    excel_path = os.path.join(args.output, "invoice_report.xlsx")
    export_to_excel(results, excel_path)

    # Export to JSON (for dashboard)
    json_path = os.path.join(args.output, "invoice_data.json")
    export_to_json(results, json_path)

    # Print summary
    total = sum(inv.get("total_amount", 0) or 0 for inv in results if "error"
    overdue = sum(1 for inv in results if inv.get("is_overdue"))
    print(f"\n{'=' * 55}")
    print(f" 📊 SUMMARY")
    print(f"      Processed: {len(results)} invoice(s)")
    print(f"      Total:      €{total:,.2f}")
    print(f"      Overdue:    {overdue}")
    print(f"{'=' * 55}\n")

if __name__ == "__main__":
    main()

```